

Step 1 — A 2-Node Graph With No Branching, Just to Prove the Mechanics — Solution

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Basic Version

Approach 1 — the direct way, a generic field name

```
# state.py
from typing import TypedDict

class GraphState(TypedDict):
    message: str
```

```
# nodes.py
from state import GraphState

def node_a(state: GraphState) -> dict:
    return {"message": state["message"] + " -> touched by A"}

def node_b(state: GraphState) -> dict:
    return {"message": state["message"] + " -> touched by B"}
```

```
# graph.py
from langgraph.graph import StateGraph, START, END
from state import GraphState
from nodes import node_a, node_b

def build_graph():
    builder = StateGraph(GraphState)
    builder.add_node("a", node_a)
    builder.add_node("b", node_b)
    builder.add_edge(START, "a")
    builder.add_edge("a", "b")
    builder.add_edge("b", END)
    return builder.compile()
```

```
# main.py
from graph import build_graph

graph = build_graph()
result = graph.invoke({"message": "start"})
print(result)
```

Expected output:

```
{'message': 'start -> touched by A -> touched by B'}
```

This proves the mechanics work — state really does flow node A, then node B, in order, and each node's returned dict really does merge into the running state. `message` is a fine name for a one-off exercise, but it's vague about what it will actually hold once real nodes replace these placeholders in Step 2.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — typed, with a manual trace comment showing the point of the exercise

```
# main.py
from graph import build_graph

graph = build_graph()

# Trace by hand before running:
#   after node_a: {"message": "start -> touched by A"}
#   after node_b: {"message": "start -> touched by A -> touched by B"}
result = graph.invoke({"message": "start"})
print(result)
assert result["message"] == "start -> touched by A -> touched by B"
```

Expected output:

```
{'message': 'start -> touched by A -> touched by B'}
```

The `assert` isn't required by the exercise, but it's a cheap, permanent way to record that your hand-trace and the code actually agreed — worth keeping as the graph gets more complex in later steps, when a silent mismatch would be much harder to spot by eye.

Difference from Basic: same graph, same 2 nodes, same edge — the only addition is writing the trace down as a comment (and a real assertion) instead of just eyeballing the printed output once and moving on. This is the actual instruction in the README's Step 1 ("by hand, trace what the state looks like after each node runs... check that your own understanding matches what the code actually does") turned into something you keep, not just something you do once in your head.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Approach 1 — the field named for what it means project-wide, with ownership documented

```
# state.py
from typing import TypedDict

class GraphState(TypedDict):
    task: str # the user's original request – set once by the caller before graph
              # every later step's nodes read this, none of them overwrite it
```

```
# nodes.py
from state import GraphState

def node_a(state: GraphState) -> dict:
    """Placeholder node A – Step 2 replaces this with real Worker logic."""
    return {"task": state["task"] + " -> touched by A"}

def node_b(state: GraphState) -> dict:
    """Placeholder node B – Step 2 replaces this with real Worker logic."""
    return {"task": state["task"] + " -> touched by B"}
```

```
# graph.py – identical structure to Intermediate, just the field name changed even
from langgraph.graph import StateGraph, START, END
from state import GraphState
from nodes import node_a, node_b

def build_graph():
    builder = StateGraph(GraphState)
    builder.add_node("a", node_a)
    builder.add_node("b", node_b)
    builder.add_edge(START, "a")
    builder.add_edge("a", "b")
```

```
builder.add_edge("b", END)
return builder.compile()
```

```
# main.py
from graph import build_graph

graph = build_graph()
result = graph.invoke({"task": "start"})
print(result)
assert result["task"] == "start -> touched by A -> touched by B"
```

Expected output:

```
{'task': 'start -> touched by A -> touched by B'}
```

Nothing about the graph's behavior changed from Intermediate — only the field's name and the ownership comment. That's the entire point: this rename is free right now, on a 2-node graph nobody else depends on yet, and it's the exact name Step 2's real Worker nodes, Step 3's Retriever, and Step 5's Approval agent all read from without ever needing to rename it again.

Difference from Intermediate: Intermediate proves the mechanics and records the trace. Advanced additionally treats the state shape as the one artifact from this step that survives into every later one, naming its field for its eventual project-wide meaning and documenting who's allowed to set it — a decision that costs nothing to make correctly here and would cost a multi-file rename to fix later.

Which one should you actually write? The Advanced version's field name and comment, paired with Intermediate's hand-trace-then-assert habit. The graph-building code itself genuinely doesn't get more complex than this at any level — this step's whole lesson is trusting the mechanics, and the only decision here with real downstream weight is what you call the one field everything else will eventually read.